

Task 2 — Query & Data Modeling

Olist Brazilian E-Commerce Dataset | Stockbit/Bibit Technical Test

1. Data Cleansing Rules

All cleansing logic is implemented in SQL and applied before data is loaded into the warehouse. There is no pre-processing step in Python — DuckDB reads the source Parquet files directly via views and applies transformations inline.

Where to Find the Cleansing Logic

File	Layer	What it cleans
script/sql/02_load_dimensions.sql	Dimensions	Customer dedup, seller/category casing, product NULL handling, type casting
script/sql/03_load_facts.sql	Facts	Numeric casting, dirty price row filter, review dedup, timestamp casting

Cleansing Rules by Table

olist_customers_dataset

Column	Issue Found	Rule Applied
customer_unique_id	99,441 rows but only 96,096 unique real customers — customer_id is order-specific	QUALIFY ROW_NUMBER() OVER (PARTITION BY customer_unique_id) = 1 to dedup
customer_city	Mixed case (no inconsistency found, normalized for consistency)	LOWER(TRIM(...))
customer_state	—	UPPER(TRIM(...))

olist_sellers_dataset_city

Column	Issue Found	Rule Applied
seller_city	2,227 rows with inconsistent casing: "CAMPiNaS", "Rio dE JANEiRo"	LOWER(TRIM(...))
seller_state	—	UPPER(TRIM(...))

olist_products_dataset

Column	Issue Found	Rule Applied
product_category_name	610 NULL values (~1.9%)	COALESCE(..., 'unknown')
product_name_lenght (sic)	Source column has typo in name	Renamed to product_name_length in warehouse
All numeric columns	Stored as string (VARCHAR) in source	TRY_CAST(... AS INTEGER) — returns NULL on failure rather than erroring
product_weight_g etc.	3 NULL values in physical dimensions	Kept as NULL — no meaningful imputation possible

product_category_name_translation

Column	Issue Found	Rule Applied
product_category_name_english	56 rows with scrambled casing: "COMpUTeRS_acceSsoRIES"	LOWER(TRIM(...))

olist_order_reviews_dataset

Column	Issue Found	Rule Applied
review_id	827 duplicate review_ids — same review linked to 2–3 different order_ids (not exact row duplicates)	QUALIFY ROW_NUMBER() OVER (PARTITION BY review_id ORDER BY review_answer_timestamp DESC) = 1 — keep most recent
review_comment_title/message	2,022 and 9,621 rows with leading/trailing whitespace	NULLIF(TRIM(...), '') — empty strings normalized to NULL
review_creation_date	1 NULL value	Kept as NULL — lifecycle stage not reached
review_score, all dates	Stored as VARCHAR	TRY_CAST(... AS INTEGER / TIMESTAMP)

olist_order_items_dataset

Column	Issue Found	Rule Applied
price	1 row: price = 'empattigakomakosongkosong' (Indonesian text for 43.00)	WHERE TRY_CAST(price AS DECIMAL) IS NOT NULL — row dropped (1 of 112,650)
All numeric columns	Stored as VARCHAR	TRY_CAST(... AS DECIMAL(10,2))

olist_orders_dataset

Column	Issue Found	Rule Applied
order_approved_at	160 NULLs (0.2%)	Valid — order not yet approved. Kept as NULL in accumulating snapshot
order_delivered_*	1,783–2,965 NULLs	Valid — delivery not yet completed. NULLable FK in fct_order_lifecycle
All timestamp columns	Stored as VARCHAR	TRY_CAST(... AS TIMESTAMP)

DQC — Data Quality Control

After each pipeline run, automated DQC checks run against all dimension and fact tables. Results are stored in the dqc_results table inside the DWH. 35 checks in total — all must PASS before the warehouse is considered ready.

DQC File	Scope	Checks Performed
script/sql/04_dim_dqc.sql	Dimensions	PK uniqueness, business key uniqueness, SCD2 integrity (max 1 is_current per natural key, end > start), referential coverage, expected row counts
script/sql/05_fct_dqc.sql	Facts	Natural key uniqueness, FK integrity against all dims, measure sanity (price > 0, review_score in [1,5], revenue ≥ 0), derived measure integrity (total = price + freight)

2. Data Warehouse Design

Methodology: Kimball Dimensional Modeling

The warehouse follows the Kimball methodology with a star schema. Business processes are modeled as fact tables surrounded by shared dimension tables. All transformations follow the principle of conforming dimensions — `dim_date`, `dim_seller`, and `dim_product` are shared across all three fact tables.

Fact Dimension (SCD1) Dimension_hist (SCD2) Static / Junk PK Primary Key FK Foreign Key M Measure DD Degenerate Dim T1 SCD Type 1
 T2 SCD Type 2 NA Non-additive SA Semi-additive

dim_customer

SCD TYPE 1 ONLY

PK	customer_key	INT
BK	customer_unique_id	VARCHAR
BK	customer_id	VARCHAR T1
	customer_zip_code	VARCHAR T1
	customer_city	VARCHAR T1
	customer_state	VARCHAR T1

CLUSTER: customer state

dim_seller

SCD TYPE 1 — CURRENT VALUES

PK	seller_key	INT
BK	seller_id	VARCHAR
	seller_zip_code	VARCHAR T1
	seller_city	VARCHAR T1
	seller_state	VARCHAR T1

CLUSTER: seller state, seller city

dim_seller_hist

SCD TYPE 2 — LOCATION HISTORY

PK	seller_hist_key	INT
BK	seller_id	VARCHAR
	seller_zip_code	VARCHAR T1
	seller_city	VARCHAR T2
	seller_state	VARCHAR T2
	scd_start_date	DATE
	scd_end_date	DATE
	is_current	BOOL

PARTITION: scd start date (MONTH)

CLUSTER: seller id, is current

fct_order_items

TRANSACTION GRAIN - 1 ROW PER ORDER ITEM

PK	order_item_key	INT
FK	order_date_key	→ dim_date
FK	customer_key	→ dim_customer
FK	seller_key	→ dim_seller
FK	product_key	→ dim_product
FK	order_flags_key	→ dim_order_flags

DD	order_id	VARCHAR
DD	order_item_id	INT

M	price	NUMERIC
M	freight_value	NUMERIC
M	total_item_value	NUMERIC
	shipping_limit_date	DATE

All measures: fully additive

PARTITION: order date (DAY)

CLUSTER: customer key, seller key, product key

dim_date

STATIC SPINE - NO SCD

PK	date_key	INT (YYYYMMDD)
	full_date	DATE
	year / quarter / month	INT
	month_name / day_name	VARCHAR
	week_of_year	INT
	day_of_week	INT
	is_weekend	BOOL

No partition needed (~730 rows)

dim_product

SCD TYPE 1 — CURRENT VALUES

PK	product_key	INT
BK	product_id	VARCHAR
	category_name	VARCHAR T1
	category_name_english	VARCHAR T1
	product_weight_g	INT T1
	product_length_cm	INT T1
	product_height_cm	INT T1
	product_width_cm	INT T1
	product_photos_qty	INT T1

CLUSTER: category name english

dim_product_hist

SCD TYPE 2 — CATEGORY HISTORY

PK	product_hist_key	INT
BK	product_id	VARCHAR
	category_name	VARCHAR T2
	category_name_english	VARCHAR T2
	product_weight_g	INT T1
	scd_start_date	DATE
	scd_end_date	DATE
	is_current	BOOL

PARTITION: scd start date (MONTH)

CLUSTER: product id, is current

dim_payment_type

STATIC LOOKUP - 5 ROWS

PK	payment_type_key	INT
BK	payment_type	VARCHAR
	payment_type_desc	VARCHAR

No partition - used by fct_order_payments (extended)

dim_order_flags

JUNK DIMENSION - MAX 32 ROWS

PK	order_flags_key	INT
	order_status	VARCHAR (8 vals)
	is_late_delivery	BOOL
	is_multi_item	BOOL

No partition - $8 \times 2 \times 2 = 32$ combinations max**fct_order_lifecycle**

ACCUMULATING SNAPSHOT - 1 ROW PER ORDER

PK	order_lifecycle_key	INT
FK	purchase_date_key	→ dim_date
FK	approved_date_key	→ dim_date
FK	carrier_date_key	→ dim_date
FK	delivered_date_key	→ dim_date
FK	estimated_date_key	→ dim_date
FK	customer_key	→ dim_customer
FK	order_flags_key	→ dim_order_flags

DD	order_id	VARCHAR
M	approval_delay_hours	NUMERIC SA
M	carrier_pickup_delay_hrs	NUMERIC SA
M	delivery_delay_days	NUMERIC SA
M	vs_estimated_days	NUMERIC SA

All measures: semi-additive (use AVG, NOT SUM)

PARTITION: purchase date (DAY)

CLUSTER: customer key, order flags key

fct_seller_monthly_snapshot

PERIODIC SNAPSHOT - 1 ROW PER SELLER PER MONTH

PK	snapshot_key	INT
FK	snapshot_month_key	→ dim_date
FK	seller_key	→ dim_seller

M	total_orders	INT
M	total_items_sold	INT
M	total_revenue	NUMERIC
M	total_freight	NUMERIC
M	avg_review_score	NUMERIC SA
M	avg_delivery_delay_days	NUMERIC SA

total_* additive across sellers - avg_* semi-additive

PARTITION: snapshot month key (MONTH)

CLUSTER: seller key

Schema Overview

Table	Type	Grain	Rows
fct_order_items	Fact — Transaction	1 row per order item	112,649
fct_order_lifecycle	Fact — Accumulating Snapshot	1 row per order	99,441
fct_seller_monthly_snapshot	Fact — Periodic Snapshot	1 row per seller per month	16,441
dim_date	Dimension — Static	1 row per calendar date (2015–2022)	2,922
dim_customer	Dimension — SCD Type 1	1 row per real customer (customer_unique_id)	96,096
dim_seller	Dimension — SCD Type 1	1 row per seller (current values)	3,095
dim_seller_hist	Dimension — SCD Type 2	1 row per seller version	3,095
dim_product	Dimension — SCD Type 1	1 row per product (current values)	32,951
dim_product_hist	Dimension — SCD Type 2	1 row per product version	32,951
dim_payment_type	Dimension — Static Lookup	1 row per payment type	5
dim_order_flags	Dimension — Junk	1 row per flag combination	32

Fact Table Types

Transaction Grain — fct_order_items

Records one row per order item. This is the primary revenue fact. All measures are fully additive: price, freight_value, and total_item_value can be safely summed across any dimension.

Accumulating Snapshot — fct_order_lifecycle

Records one row per order, updated as the order progresses through lifecycle stages (purchase → approved → carrier pickup → delivered). Each stage has its own FK to dim_date. Delay measures are semi-additive: meaningful as AVG per seller or region, not as SUM.

Measure	Type	Formula
approval_delay_hours	Semi-additive	Minutes from purchase to approval / 60
carrier_pickup_delay_hours	Semi-additive	Minutes from approval to carrier pickup / 60

Measure	Type	Formula
delivery_delay_days	Semi-additive	Minutes from carrier pickup to customer delivery / 1440
vs_estimated_days	Semi-additive	Minutes from estimated to actual delivery / 1440 (positive = late)

Periodic Snapshot — fct_seller_monthly_snapshot

Records one row per seller per calendar month, generated at the end of each month. total_* measures are additive across sellers but not across time (would double-count). avg_* measures are semi-additive.

SCD Type Strategy

Slowly Changing Dimensions are handled at the column level — SCD Type 1 and Type 2 can coexist within the same dimension.

Dimension	SCD Type 1 Columns	SCD Type 2 Columns	Rationale
dim_customer	zip_code, city, state	—	No change history in source data; customer_id is order-specific
dim_seller / dim_seller_hist	seller_zip_code	seller_city, seller_state	Zip is treated as always-current; city/state changes are tracked for accurate historical regional analysis
dim_product / dim_product_hist	weight_g, dimensions (cm)	category_name, category_name_english	Physical dimensions overwritten in-place; category changes affect historical category revenue reporting
dim_date, dim_payment_type, dim_order_flags	—	—	Static tables, no SCD needed

SCD Type 1 update logic: when a Type 1 attribute changes, ALL historical rows for that natural key are updated in-place. SCD Type 2 logic: a new row is inserted with a new scd_start_date and the old row's scd_end_date is closed. At any point in time, only one row per natural key has is_current = TRUE.

Idempotency

All SQL scripts are idempotent — running the pipeline multiple times produces identical results:

- Static tables (dim_date, dim_payment_type, dim_order_flags): INSERT OR IGNORE
- SCD Type 1 dimensions: MERGE on business key (UPDATE on change, INSERT if new)

- SCD Type 2 hist tables: 3-step — UPDATE SCD1 cols across all rows, CLOSE changed current records, INSERT new current record
- Fact tables: DELETE on natural key + INSERT fresh

3. Entity Relationship Diagram (ERD)

The ERD below shows the full star schema of the Olist Data Warehouse including all fact and dimension tables, foreign key relationships, and SCD type annotations.

[ERD SCREENSHOT PLACEHOLDER]

Insert ERD screenshot here

The ERD was designed using the visualization built during development. Key relationships:

- All 3 fact tables share dim_date (multiple FKs per fact for fct_order_lifecycle — one per lifecycle stage)
- fct_order_items joins to dim_customer, dim_seller, dim_product, dim_order_flags
- dim_seller_hist and dim_product_hist mirror their SCD1 counterparts with added scd_start_date, scd_end_date, is_current
- dim_order_flags is a junk dimension combining order_status (8 values) × is_late_delivery (bool) × is_multi_item (bool) = 32 max rows

4. SQL Scripts

Repository Structure

Path	Description
script/sql/01_ddl.sql	DDL — CREATE TABLE IF NOT EXISTS for all 11 DWH tables + sequences + dqc_results table
script/sql/02_load_dimensions.sql	Idempotent dimension loading with cleansing CTEs, MERGE for SCD1, 3-step pattern for SCD2
script/sql/03_load_facts.sql	Idempotent fact loading with cleansing CTEs, DELETE + INSERT for all 3 fact types
script/sql/04_dim_dqc.sql	18 DQC checks for all dimension tables — results written to dqc_results
script/sql/05_fct_dqc.sql	17 DQC checks for all fact tables — FK integrity, measure sanity, uniqueness
script/py/run_pipeline.py	Python orchestrator — wires raw Parquet views and executes SQL files in order
db/olist_dwh.zip	Pre-built DuckDB warehouse file (compressed). Unzip to get olist_dwh.duckdb

End-to-End Steps to Run the Pipeline

Prerequisites

- Python 3.8 or later
- `pip install duckdb pyarrow`
- Parquet source files in `olist_dataset/` folder at project root

Step 1 — Clone the repository

```
git clone https://github.com/iqbalmohamad/stockbit-olist-dwh-test.git
cd stockbit-olist-dwh-test
```

Step 2 — Place source data

Extract the Olist Parquet files into the `olist_dataset/` folder at project root:

```
stockbit-olist-dwh-test/
├─ olist_dataset/
│   ├── olist_orders_dataset.parquet
│   ├── olist_customers_dataset.parquet
│   └─ ... (all 8 parquet files)
```

Step 3 — Run the pipeline

```
python script/py/run_pipeline.py
```

The pipeline will:

- Wire all 8 Parquet files as DuckDB views
- Execute 01_ddl.sql → create all tables and sequences
- Execute 02_load_dimensions.sql → cleanse and load all dimensions
- Execute 03_load_facts.sql → cleanse and load all fact tables
- Execute 04_dim_dqc.sql and 05_fct_dqc.sql → run 35 DQC checks
- Run CHECKPOINT to flush WAL — required before API can open in read-only mode

Expected output:

```
Connecting to: ../db/olist_dwh.duckdb
▶ Creating raw views from parquet files
  ✓ 8 views created
▶ 01_ddl.sql ✓ 21 statements executed
▶ 02_load_dimensions.sql ✓ 15 statements executed
▶ 03_load_facts.sql ✓ 6 statements executed
▶ 04_dim_dqc.sql ✓ 18 statements executed
▶ 05_fct_dqc.sql ✓ 17 statements executed
DQC: 35/35 PASS
Done. DWH available at: ../db/olist_dwh.duckdb
```

Using the Pre-built DWH (optional)

If you prefer not to run the pipeline, a pre-built DWH is included:

- Unzip db/olist_dwh.zip → db/olist_dwh.duckdb
- Open with DBeaver, TablePlus, or the DuckDB CLI to query directly
- Or proceed to the API step below — api.py will read from db/olist_dwh.duckdb automatically

The DWH is stored as a ZIP because the raw .duckdb file exceeds GitHub's 100MB file size limit. After unzipping, the file is approximately 150MB.

5. Simple API

Overview

A FastAPI application that serves analytics data from the DWH over HTTP. The API connects to DuckDB in read-only mode, which allows it to run concurrently with other tools (DBeaver, pipeline re-runs).

Detail	Value
Framework	FastAPI + Uvicorn
Database	DuckDB (read-only connection)
Auto-generated docs	http://localhost:8000/docs (Swagger UI)
Source file	script/py/api.py

End-to-End Steps to Run the API

Prerequisites

- Pipeline has been run at least once (db/olist_dwh.duckdb exists), OR pre-built DWH has been unzipped
- `pip install fastapi uvicorn`

Step 1 — Start the API server

```
python script/py/api.py
```

Expected output:

```
Starting Olist DWH API...
Swagger docs: http://localhost:8000/docs
INFO:      Uvicorn running on http://0.0.0.0:8000
```

Step 2 — Open Swagger UI or use Postman

Option A — Browser: navigate to <http://localhost:8000/docs> to see all endpoints with interactive testing.

Option B — Postman: import from the OpenAPI spec URL:

```
http://localhost:8000/openapi.json
```

API Endpoints & Test Cases

GET /health

Health check — returns API status and row counts for all DWH tables.

```
GET http://localhost:8000/health
```

Expected response:

```
{ "status": "ok",  
  "db_path": ".../db/olist_dwh.duckdb",  
  "row_counts": {  
    "dim_date": 2922, "dim_customer": 96096,  
    "fct_order_items": 112649, ... } }
```

GET /sellers

List sellers from dim_seller with optional filters. Returns current SCD Type 1 values.

```
GET http://localhost:8000/sellers?state=SP&limit=10  
GET http://localhost:8000/sellers?city=sao+paulo&limit=5
```

Query parameters:

Parameter	Type	Description	Example
state	string (optional)	Filter by Brazilian state code	SP, RJ, MG
city	string (optional)	Partial match on city name	sao paulo
limit	integer (optional)	Max results, default 50, max 500	10

GET /sellers/{seller_id}/snapshot

Monthly performance for a specific seller from fct_seller_monthly_snapshot. Returns revenue, order counts, avg review score, and avg delivery delay per month.

```
GET  
http://localhost:8000/sellers/3442f8959a84dea7ee197c632cb2df15/snapshot?months=6
```

Note: avg_review_score and avg_delivery_delay_days are semi-additive — do not sum across months.

GET /orders/{order_id}/lifecycle

Stage-by-stage lifecycle for a single order from fct_order_lifecycle (accumulating snapshot). Returns delay metrics between each pipeline stage.

```
GET http://localhost:8000/orders/e481f51cbdc54678b7cc49136f2d6af7/lifecycle
```

Expected response:

```
{ "order_id": "e481f51cbdc54678b7cc49136f2d6af7",  
  "order_status": "delivered",  
  "order_purchase_date": "2017-10-02",
```

```
"approval_delay_hours": 0.18,  
"carrier_pickup_delay_hours": 56.8,  
"delivery_delay_days": 6.06,  
"vs_estimated_days": -7.11,  
"is_late": false }
```

GET /revenue/by-category

Revenue breakdown by product category from fct_order_items joined to dim_product. All measures are fully additive.

```
GET http://localhost:8000/revenue/by-category?limit=10  
GET http://localhost:8000/revenue/by-category?date_from=2017-01-01&date_to=2017-12-31&limit=5
```

Parameter	Type	Description
date_from	string YYYY-MM-DD (optional)	Filter orders from this date
date_to	string YYYY-MM-DD (optional)	Filter orders until this date
limit	integer (optional)	Max results, default 20, max 100

GET /dq

Latest DQC check results from the most recent pipeline run. Use this to verify data quality after running the pipeline.

```
GET http://localhost:8000/dqc  
GET http://localhost:8000/dqc?layer=fct  
GET http://localhost:8000/dqc?status=FAIL
```

A healthy pipeline returns 35 results, all with status: PASS. Any FAIL indicates a data quality issue that should be investigated before using the warehouse for analysis.